

Buổi 3 — Dữ liệu thời gian với pandas, polars, DuckDB

1. Mục tiêu

Sau buổi này bạn:

- Đổi được một giờ địa phương sang **UTC** (giờ chung của thế giới) bằng tay và bằng pandas. Chỉ ra được giờ nào "không tồn tại" và giờ nào "xảy ra hai lần" ở New York năm 2024.
- Chỉ ra bằng con số trên dữ liệu taxi thật: hai ngày đổi **giờ mùa hè** làm hỏng một chuỗi đếm theo giờ ra sao.
- Chọn đúng cách gộp khi đổi tần suất: **cộng** cho số lượng (số chuyến), **trung bình** hoặc **giá trị cuối** cho số đo trạng thái (nhiệt độ). Phân biệt giờ trống là "0" hay "không biết".
- Ghép hai nguồn khác múi giờ mà không tạo quan hệ giả, và có một phép thử nhanh để phát hiện ghép lệch.
- Viết hàm `chuan_hoa_thoi_gian()`. Hàm nhận dữ liệu thô bất kỳ và trả một bảng **dạng dài** ba cột `unique_id`, `ds`, `y`. Cột thời gian theo UTC, đủ mọi mốc, không dòng trùng. Hàm phải qua 11 test của bộ chấm.

Hàm này là **cột mốc M0** của khoá: sản phẩm đầu tiên bạn tự làm, và là dạng dữ liệu mà mọi buổi sau nhận vào.

2. Nhắc lại buổi trước

Buổi 1 và 2 cho những ý dùng ngay hôm nay. Mỗi ý có một ví dụ nhỏ, không cần mở lại buổi cũ.

- **Mốc cắt dữ liệu** (cutoff): lúc ra dự báo, ta chỉ biết những gì đã xảy ra *trước* mốc đó. Ví dụ: dự báo lúc 08:00 thì không được dùng lượng mưa lúc 09:00. Hôm nay nguyên tắc này có nghĩa là: giá trị gần cho giờ t chỉ được lấy từ thông tin đã có tại t .
- **Chia dữ liệu theo thời gian**. Tập huấn luyện là phần quá khứ dùng để làm mô hình. Tập kiểm tra là phần sau đó, dùng để chấm. Ví dụ: tháng 1–2 để làm, tháng 3 để chấm. Muốn cắt đúng thì trục thời gian phải đúng trước đã. Một giờ bị lặp hay bị mất là một dòng rơi nhầm phía.
- **Trung bình và phương sai**. Phương sai đo các giá trị tản ra quanh trung bình bao xa. Cách tính: lấy mỗi giá trị trừ trung bình, bình phương, rồi lấy trung bình các bình phương đó. Ví dụ 2, 4, 6: trung bình 4; độ lệch $-2, 0, 2$; bình phương 4, 0, 4; phương sai $(4 + 0 + 4) / 3 \approx 2,67$. (Có sách chia cho $n - 1$.)
- **Dữ liệu đếm** (số chuyến, số đơn hàng) chỉ nhận 0, 1, 2... Khi chia nhỏ theo nơi và giờ, rất nhiều ô bằng 0 và phương sai thường lớn hơn trung bình. Ví dụ 0, 0, 0, 1, 9: trung bình 2, phương sai $(4 + 4 + 4 + 1 + 49) / 5 = 12,4$. Hôm nay bạn sẽ thấy hơn một nửa số ô "khu vực $\times$ giờ" của taxi New York bằng 0.

Cú pháp thời gian của pandas, polars, DuckDB được tổng hợp thêm ở Phụ lục A của khoá. Tài liệu này tự đủ, không cần mở phụ lục.

3. Trạng thái đầu buổi

Sau `cd lab && make up`. Hai từ cần biết để đọc bảng: **Parquet** là một định dạng tệp lưu bảng theo từng cột, có nén, đọc nhanh hơn CSV. **sha256** là "dấu vân tay" của một tệp: tệp đổi một byte là chuỗi sha256 đổi hẳn. Bảng ghi 12 ký tự đầu để bạn so với máy mình.

Hạng mục	Trạng thái
Dữ liệu	<code>du-lieu/raw/nyc-tlc-yellow-2024-03/yellow_tripdata_2024-03.parquet</code> — 3.582.628 chuyển, sha256 <code>2d4cdc8fb967</code>
	<code>du-lieu/raw/nyc-tlc-yellow-2024-11/yellow_tripdata_2024-11.parquet</code> — 3.646.369 chuyển, sha256 <code>5ef321876de5</code>
	<code>du-lieu/raw/nyc-tlc-zone-lookup/taxi_zone_lookup.csv</code> — 265 khu vực, sha256 <code>1a99e1050922</code>
	<code>du-lieu/raw/open-meteo-new-york-2024-03-11/...-utc.csv</code> — thời tiết theo giờ 01/3 → 30/11/2024, UTC , sha256 <code>fd75dc93e76d</code>
	bài tập: taxi tháng 1/2024 (sha256 <code>c4d59da7bbc8</code>) và thời tiết New York tháng 1/2024 UTC (sha256 <code>117856cb5284</code>)
Môi trường	Python 3.12; pandas 3.0.5, polars 1.44.2, duckdb 1.5.5, pyarrow 25.0.1 (<code>lab/00-nen/pyproject.toml</code>)
<code>code/thoi_gian.py</code>	<code>chuan_hoa_thoi_gian</code> , <code>doc_chuyen_taxi</code> , <code>dem_chuyen_theo_gio</code> , <code>doc_thoi_tiet</code> , <code>ghep_thoi_tiet</code> , <code>gio_nong_nhat</code>
<code>code/bam_gio.py</code>	bấm giờ đọc + đếm theo giờ bằng pandas, polars, DuckDB
Đang cố tình sai	thời gian để naive (không ghi múi giờ); đếm theo giờ (<code>resample</code> , mục 4.3) trên giờ địa phương; không bỏ dòng trùng; thời tiết (UTC) ghép thẳng với giờ taxi (giờ New York)
Triệu chứng	tháng 3 có một giờ 0 chuyển ; tháng 11 có giờ 01:00 9.869 chuyển (gần gấp đôi thường lệ); <code>gio_nong_nhat()</code> trả 20 — New York "nóng nhất lúc 8 giờ tối"
<code>make check</code> lúc này	ĐỎ: 10/11 test hỏng

`make check` là lệnh chạy bộ chấm: 11 hàm test tự động gọi code của bạn và so với kết quả đúng. ĐỎ nghĩa là còn test hỏng.

4. Lý thuyết

Từ mới trong buổi

Thuật ngữ	Nghĩa một câu	Ví dụ
UTC	Giờ phối hợp quốc tế: một đồng hồ chung cho cả thế giới, chạy đều, không bao giờ vặn theo mùa.	07:00 sáng ở Hà Nội là 00:00 UTC.
hậu tố Z	Chữ Z cuối một giờ nghĩa là "giờ này tính theo UTC"; giống hệt viết <code>+00:00</code> .	<code>07:30Z</code> = 07:30 UTC = 14:30 Hà Nội.
offset (độ lệch múi giờ)	Giờ địa phương lệch UTC bao nhiêu giờ.	Hà Nội +7; New York mùa đông −5, mùa hè −4.

Thuật ngữ	Nghĩa một câu	Ví dụ
múi giờ (time zone), IANA	Tên một vùng dùng chung luật giờ, ví dụ <code>America/New_York</code> . IANA là tổ chức giữ bảng tra múi giờ mà mọi máy tính dùng.	Bảng IANA ghi: New York đổi offset hai lần mỗi năm.
giờ mùa hè (DST, daylight saving time)	Một số nơi vận đồng hồ nhanh 1 giờ vào mùa hè, rồi vận lại vào mùa thu.	New York 2024: nhanh lên ngày 10/3, lùi lại ngày 3/11.
EST / EDT	Giờ chuẩn miền Đông nước Mỹ (UTC−5) và giờ mùa hè miền Đông (UTC−4).	00:00 EST = 05:00Z; 00:00 EDT = 04:00Z.
naive / aware	Thời điểm không ghi múi giờ (naive) hoặc có ghi (aware).	<code>2024-03-10 01:30</code> là naive; <code>2024-03-10 01:30-05:00</code> là aware.
NaN, NaT	Ô "không biết" của pandas: NaN cho số (Not a Number), NaT cho thời gian (Not a Time).	Nhiệt độ lúc 01:00 không đo → NaN, khác với 0 độ.
tần suất, <code>resample</code>	Tần suất là khoảng cách giữa hai mốc liên nhau. <code>resample</code> đổi tần suất, ví dụ gộp 60 phút thành 1 giờ.	Đếm chuyển theo phút → cộng lại theo giờ.
<code>closed</code> , <code>label</code>	Khi gộp: đầu nào của khoảng được tính vào (<code>closed</code>), và khoảng được đặt tên bằng mốc đầu hay mốc cuối (<code>label</code>).	Khoảng [9:00, 10:00) tên "9:00": chứa 9:00, không chứa 10:00.
chuỗi đều / chuỗi không đều	Chuỗi đều: các mốc cách nhau bằng nhau. Không đều: lúc dày lúc thưa.	Chuyến taxi đến lúc 00:10, 00:50, 02:20 là không đều.
ghép as-of (<code>merge_asof</code>)	Ghép mỗi dòng với giá trị gần nhất đã có ở trước (hoặc đúng) thời điểm của dòng đó.	Đơn hàng lúc 10:07 lấy giá cập nhật lúc 10:05, không lấy giá lúc 10:10.
định dạng dài / định dạng rộng	Dạng dài: mỗi dòng một (chuỗi, mốc, giá trị). Dạng rộng: mỗi chuỗi một cột.	2 khu vực × 3 giờ: dạng dài 6 dòng; dạng rộng 3 dòng × 2 cột.
<code>unique_id</code> , <code>ds</code> , <code>y</code>	Ba cột của dạng dài: tên chuỗi, thời điểm, giá trị cần dự báo.	<code>("161", 2024-03-01 05:00Z, 412)</code> .
backtest	Giả vờ đứng ở nhiều mốc cắt trong quá khứ, dự báo, rồi so với cái đã thật sự xảy ra.	Đứng ở 4 thứ Hai của tháng 3, mỗi lần dự báo 24 giờ tới.

4.1 UTC, múi giờ và giờ mùa hè: một con số giờ chưa đủ để biết "khi nào"

Vấn đề. Taxi New York ghi giờ đón khách theo đồng hồ New York. Dữ liệu thời tiết của buổi ghi theo UTC. Muốn hỏi "lúc trời mưa có nhiều khách hơn không", ta phải đặt hai bảng lên **cùng một trục thời gian**. Mục này dạy trục đó.

UTC và chữ Z. UTC (giờ phối hợp quốc tế) là một đồng hồ chung cho cả thế giới. Nó chạy đều, không bao giờ vận nhanh hay chậm theo mùa. Giờ ở mọi nơi khác được nói bằng "lịch UTC bao nhiêu", gọi là **offset**. Hà Nội là UTC+7: khi UTC chỉ 00:00 thì đồng hồ Hà Nội chỉ 07:00. Khi viết một giờ UTC, người ta hay thêm chữ **Z** vào cuối. `07:30Z` đọc là "07:30 giờ UTC". Viết đầy đủ, `2024-03-10T07:30:00Z` giống hệt `2024-03-10T07:30:00+00:00`. Trong tài liệu này, "05Z" là viết gọn của "05:00Z".

Giờ mùa hè ở New York. Múi giờ `America/New_York` có hai offset. Mùa đông là **EST**, UTC−5. Mùa hè là **EDT**, UTC−4: đồng hồ được vận nhanh thêm 1 giờ để trời sáng muộn hơn theo giờ đồng hồ. Năm 2024 có hai lần vận:

- **Chủ nhật 10/3:** lúc đồng hồ sắp chỉ 02:00 EST, người ta vận nó lên 03:00 EDT. Các giờ 02:00–02:59 **không tồn tại**.

- **Chủ nhật 3/11:** lúc đồng hồ sắp chỉ 02:00 EDT, người ta vặn nó lùi về 01:00 EST. Các giờ 01:00–01:59 **xảy ra hai lần**.

Vậy từ 10/3/2024 đến 3/11/2024 New York lệch UTC 4 giờ. Ngoài khoảng đó, New York lệch UTC 5 giờ.

Trực giác. Hình dung một đồng hồ treo tường và một đồng hồ bấm giờ chạy mãi không dừng. Đồng hồ bấm giờ là UTC. Hai lần một năm, có người leo lên vặn đồng hồ treo tường. Mùa xuân vặn nhanh: một vạch trên mặt đồng hồ bị nhảy qua. Mùa thu vặn chậm: một vạch bị đi qua hai lần. Một ghi chú chỉ nói "01:30 theo đồng hồ treo tường" thì không đủ để biết đó là lần nào.

Ví dụ số nhỏ – tự tính tay. Đổi giờ New York sang UTC: **cộng 5 giờ nếu đang EST, cộng 4 giờ nếu đang EDT** (vì offset âm, trừ offset tức là cộng). Tính mẫu vài dòng:

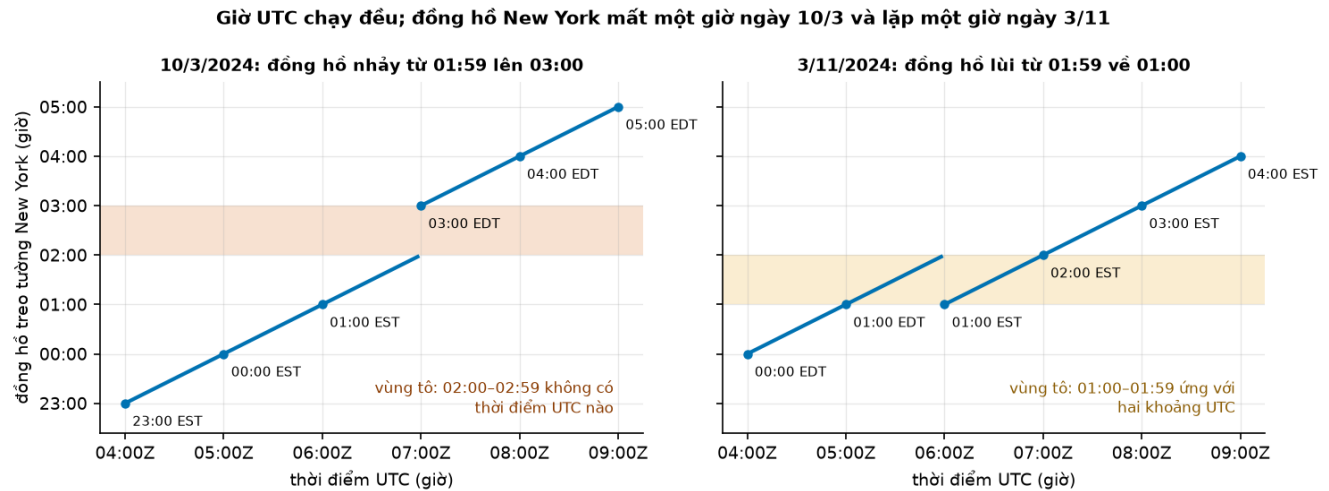
- 00:00 ngày 10/3 là EST (chưa tới giờ vặn): $00:00 + 5 = \mathbf{05:00Z}$.
- 03:00 ngày 10/3 là EDT (đã vặn): $03:00 + 4 = \mathbf{07:00Z}$. Phút cuối của EST là 01:59 + 5 = 06:59Z. Phút ngay sau đó, 07:00Z, đồng hồ đã chỉ 03:00. Không có thời điểm UTC nào để đồng hồ chỉ 02:30.
- 01:30 ngày 3/11 lần thứ nhất là EDT: $01:30 + 4 = \mathbf{05:30Z}$. Một giờ sau, đồng hồ lùi và chỉ 01:30 lần nữa, lúc này là EST: $01:30 + 5 = \mathbf{06:30Z}$.

Làm như vậy cho mọi giờ từ 00:00 tới 04:00 của hai ngày (kiểm lại bằng `dap-an/vi_du_nho.py`):

Đồng hồ New York	Chủ nhật 10/3/2024 → UTC	Chủ nhật 3/11/2024 → UTC
00:00	05:00Z (EST)	04:00Z (EDT)
01:00	06:00Z (EST)	05:00Z (EDT) và 06:00Z (EST)
01:30	06:30Z (EST)	05:30Z (EDT) và 06:30Z (EST)
02:00	không tồn tại	07:00Z (EST)
02:30	không tồn tại	07:30Z (EST)
03:00	07:00Z (EDT)	08:00Z (EST)
04:00	08:00Z (EDT)	09:00Z (EST)

Đọc bảng. So hai cột với nhau.

- Cột ngày 10/3/2024: UTC đi từ 06:00Z lên thẳng 07:00Z, nhưng đồng hồ nhảy từ 01:59 lên 03:00. Ngày này chỉ có **23 giờ**.
- Cột ngày 3/11/2024: hai dòng 01:00 và 01:30 có hai đáp án. Ngày này có **25 giờ**.
- Một giờ New York không kèm offset có thể ứng với 0, 1 hoặc 2 thời điểm UTC. Giờ UTC thì luôn chỉ đúng một thời điểm.



Cách đọc hình.

1. **Trục ngang** (cả hai ô): thời điểm UTC, từ 04:00Z tới 09:00Z, mỗi vạch một giờ.
2. **Trục dọc**: đồng hồ treo tường New York chỉ mấy giờ tại thời điểm UTC đó (giờ). Vạch 23:00 là 23 giờ đêm hôm trước.
3. **Ký hiệu**: đường xanh là giờ đồng hồ New York; chấm và nhãn ghi giờ New York tại mỗi giờ UTC tròn. Dải tô màu là vùng giờ có vấn đề.
4. **Nhìn vào đâu**: ô trái, ở 07:00Z đường nhảy từ 02:00 lên 03:00, nên không điểm nào của đường nằm trong dải cam. Ô phải, ở 06:00Z đường rơi từ 02:00 về 01:00, nên dải vàng bị đường đi qua hai lần.
5. **Kết luận**: UTC chạy đều; đồng hồ New York mất một giờ ngày 10/3 và lặp một giờ ngày 3/11.

Công thức.

$$t_{\text{UTC}} = t_{\text{địa phương}} - \text{offset}(t)$$

- $t_{\text{địa phương}}$: giờ đồng hồ treo tường ở nơi ghi dữ liệu.
- $\text{offset}(t)$: độ lệch so với UTC tại đúng thời điểm đó. New York: -5 giờ (EST) hoặc -4 giờ (EDT). Hà Nội: $+7$ giờ.
- t_{UTC} : cùng thời điểm đó, đọc trên đồng hồ UTC.

Nói bằng lời. Giờ UTC bằng giờ địa phương trừ đi offset. Ví dụ 20:00 tối 9/3 ở New York có offset -5 , nên UTC là $20:00 - (-5) = 20:00 + 5 = 01:00$ ngày 10/3. Ở Hà Nội, $07:00 - (+7) = 00:00$ UTC. Chỗ khó: muốn biết offset thì phải biết thời điểm, mà muốn biết thời điểm lại cần offset. Ngày thường điều đó không sao. Riêng hai ngày đổi giờ, phép đổi ngược có **0 nghiệm** (02:30 ngày 10/3) hoặc **2 nghiệm** (01:30 ngày 3/11). Việt Nam (`Asia/Ho_Chi_Minh`) hiện không đổi giờ theo mùa: offset luôn $+7$ (bảng IANA ghi như vậy từ 13/6/1975).

Tự viết bằng NumPy. Ngày thường, đổi giờ chỉ là phép trừ:

```
import numpy as np
# datetime64[m]: kiểu thời gian của NumPy, chính xác tới phút ([m] = minute)
gio_ny = np.array(["2024-03-09T20:00", "2024-03-11T20:00"], dtype="datetime64[m]")
# np.timedelta64(1, "h") là một khoảng thời gian dài 1 giờ; nhân với -5, -4 ra offset EST, EDT
offset = np.array([-5, -4]) * np.timedelta64(1, "h")
gio_utc = gio_ny - offset          # ['2024-03-10T01:00', '2024-03-12T00:00']
```

Dòng thứ hai trong kết quả: 20:00 ngày 11/3/2024 là EDT, nên $20 + 4 = 24$, tức 00:00 ngày 12/3/2024 UTC. Ở đây ta tự gõ offset cho từng dòng. Với dữ liệu thật, đừng tự gõ: dùng bảng IANA qua thư viện.

Thư viện. pandas có hai hàm, làm hai việc khác nhau:

- `tz_localize("America/New_York")`: **gắn** múi giờ cho giá trị naive. Con số giờ giữ nguyên, chỉ thêm nhãn "đây là giờ New York". Từ đây giá trị thành aware.
- `tz_convert("UTC")`: **đổi** một giá trị aware sang múi giờ khác. Con số giờ thay đổi, thời điểm thật giữ nguyên.

Gặp giờ không tồn tại hay giờ lặp, `tz_localize` mặc định **báo lỗi** (`ValueError`) và dừng. Hai tham số chọn cách xử lý:

- `nonexistent` (giờ không tồn tại, như 02:30 ngày 10/3): `"raise"` báo lỗi; `"NaT"` đánh dấu "không biết"; `"shift_forward"` dời lên giờ hợp lệ kế tiếp (03:00).
- `ambiguous` (giờ mơ hồ, tức giờ lặp như 01:30 ngày 3/11): `"raise"` báo lỗi; `"NaT"` đánh dấu "không biết"; `"infer"` đoán theo thứ tự các dòng (lần gặp đầu là EDT, lần sau là EST); hoặc một mảng True/False, True là giờ mùa hè.

Công cụ khác xử lý khác. DuckDB (cơ sở dữ liệu chạy SQL ngay trên tệp) **không báo lỗi** mà im lặng dời giờ: 02:30 ngày 10/3/2024 thành 07:30Z, trùng với 03:30 EDT. Bảng so ba công cụ nằm ở mục "Nâng cao" cuối mục 4.

Tóm lại. Một giờ không kèm múi giờ (naive) chưa chỉ ra một thời điểm. Hãy đổi mọi thứ về UTC (có ghi múi giờ) trước khi làm bất cứ gì: đổi giờ New York thì cộng 5 (EST) hoặc 4 (EDT). Ngày 10/3/2024 mất các giờ 02:00-02:59; ngày 3/11/2024 các giờ 01:00-01:59 xảy ra hai lần.

Tự kiểm tra. (a) Cảm biến ở Hà Nội ghi 2024-07-01 07:15 (naive). Giờ UTC là mấy giờ? (b) Cảm biến ở New York ghi 2024-11-03 01:15. Giờ UTC là mấy giờ?

Đáp án (tự làm trước rồi mới đọc)

(a) Offset +7, nên $07:15 - 7 = 00:15Z$ ngày 1/7. (b) Có **hai** đáp án: $01:15 \text{ EDT} = 05:15Z$, và $01:15 \text{ EST} = 06:15Z$. Chỉ từ con số giờ, không biết là lần nào. Nhầm hay gặp: cộng 7 thay vì trừ ở câu (a), ra 14:15Z. Hãy nhớ: Hà Nội đi **trước** UTC, nên giờ UTC phải **nhỏ hơn** giờ Hà Nội.

4.2 Giờ mùa hè trong dữ liệu taxi thật

Vấn đề. Tệp taxi có ghi múi giờ không? Từ điển dữ liệu của cơ quan quản lý taxi New York chỉ viết cột `trip_pickup_datetime` là "ngày giờ lúc đồng hồ tính tiền được bật". Không nhắc múi giờ. Tệp Parquet lưu cột này ở kiểu thời gian không múi giờ. Ta phải tìm bằng chứng trong chính dữ liệu.

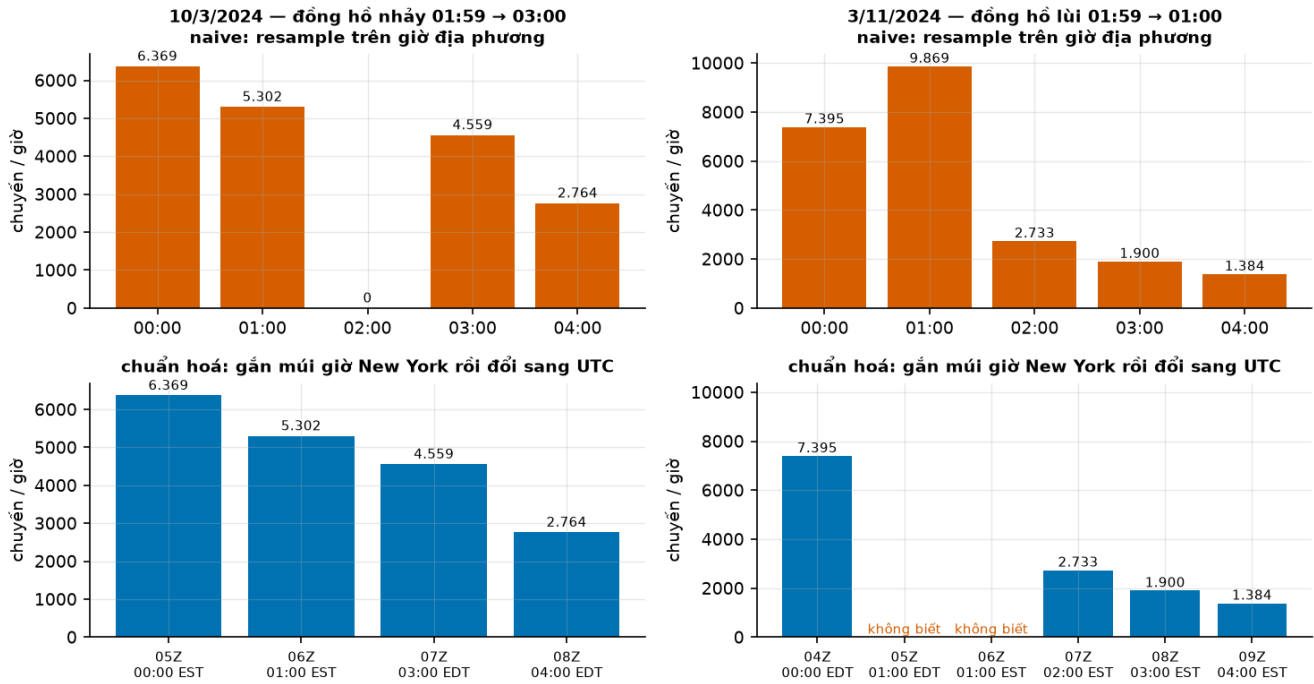
Thực giác. Nếu cột này là giờ đồng hồ New York, ngày 10/3/2024 sẽ **không có chuyến nào** đón lúc 02:00-02:59, vì giờ đó không tồn tại. Và ngày 3/11/2024, giờ 01:00-01:59 sẽ chứa khách của **hai** giờ thật. Nếu cột là UTC thì không có hai dấu hiệu này.

Ví dụ số nhỏ — tự tính tay. Bốn chuyến đêm 10/3, giờ đón naive: 01:20, 01:50, 03:10, 03:40.

- Đếm theo giờ trên chính con số naive: giờ 01:00 có 2 chuyến, giờ 02:00 có **0**, giờ 03:00 có 2. Ba dòng, một dòng bằng 0.
- Đổi sang UTC trước (bảng mục 4.1): $01:20 + 5 = 06:20Z$; $01:50 + 5 = 06:50Z$; $03:10 + 4 = 07:10Z$; $03:40 + 4 = 07:40Z$. Đếm theo giờ UTC: 06Z có 2, 07Z có 2. **Hai giờ liền nhau, không có giờ 0.**
- Chuyến 01:50 và 03:10 nhìn cách nhau 80 phút, thật ra chỉ cách nhau 20 phút ($06:50Z \rightarrow 07:10Z$).

Ngày 3/11 làm ngược lại. Bốn chuyến ghi 01:10, 01:40, 01:20, 01:50 có thể là hai chuyến EDT và hai chuyến EST. Đếm naive thì giờ 01:00 có 4 chuyến, gấp đôi một giờ bình thường. Chỉ từ giờ đón, không có cách nào biết chuyến nào thuộc lần nào.

Resample giờ địa phương naive sinh giờ 0 chuyển giả (tháng 3) và gộp hai giờ thật (tháng 11)



Cách đọc hình.

- Trục ngang:** hàng trên là giờ đồng hồ New York 00:00–04:00. Hàng dưới là giờ UTC, dòng thứ hai của nhãn ghi giờ New York tương ứng. Ví dụ "05Z / 00:00 EST" nghĩa là 05:00Z, lúc đồng hồ New York chỉ 00:00 EST ($0 + 5 = 5$).
- Trục dọc:** số chuyến đón trong mỗi giờ (chuyến/giờ).
- Ký hiệu:** cột cam là cách của `code/` (đếm trên giờ naive). Cột xanh là sau khi gắn múi giờ New York rồi đổi sang UTC. Chữ "không biết" đánh dấu giờ không đếm được.
- Nhìn vào đâu:** ô trên bên trái, cột 02:00 bằng 0. Ô trên bên phải, cột 01:00 cao vọt. Hàng dưới, hai chỗ đó.
- Kết luận:** đếm trên giờ naive sinh một giờ 0 chuyển giả (tháng 3) và gộp hai giờ thật làm một (tháng 11).

Ô trên bên trái: 0 chuyến lúc 02:00 vì giờ đó không tồn tại, không phải vì không ai đi taxi. Ô trên bên phải: giờ 01:00 có **9.869** chuyến, trong khi cũng giờ đó Chủ nhật tuần sau chỉ có **5.318**: hai giờ thật bị gộp làm một.

Hàng dưới là sau khi đổi sang UTC:

- Ô trái: các giờ UTC liên nhau từ 05:00Z tới 08:00Z, không còn giờ ảo.
- Ô phải: có 6 giờ UTC, nhưng 05:00Z và 06:00Z ghi "không biết".
- Lý do: một chuyến ghi 01:xx có thể rơi vào 05:00Z (nếu là EDT) hoặc 06:00Z (nếu là EST), như bảng mục 4.1.

Không tách được, nên hàm của đáp án làm ba việc:

- Bỏ các dòng mơ hồ đó, và ghi số dòng đã bỏ vào `df.attrs`. `attrs` là một từ điển nhỏ gắn kèm bảng pandas để ghi chú, ví dụ `{"bo_vi_dst": 9869, "trung_lap": 0}`.
- Tính xem mỗi dòng bỏ **có thể** thuộc mốc UTC nào. Gắn múi giờ cho nó hai lần: một lần hiểu là EDT (`ambiguous` là mảng toàn True), một lần hiểu là EST (toàn False). Đổi sang UTC, làm tròn xuống giờ, được 05:00Z và 06:00Z.
- Ghi các mốc đó là NaN, không ghi 0.

"Không có chuyến" (0) và "không biết có bao nhiêu chuyến" (NaN) là hai điều khác nhau. Điền 0 là bịa ra một giờ vắng khách.

Dấu vết khác trong dữ liệu. Thời lượng chuyến = giờ trả – giờ đón. Tính trên giờ naive thì hai ngày đổi giờ cho số sai:

- 10/3: đón 01:50 EST, trả 03:05 EDT. Thật ra chuyến dài 15 phút (06:50Z → 07:05Z). Trên giờ naive, 01:50 → 03:05 là 75 phút: **đur đúng một giờ**, vì giờ 02:xx bị nhảy qua.
- 3/11: đón 01:50 EDT, trả 01:10 EST. Thật ra dài 20 phút (05:50Z → 06:10Z). Trên giờ naive, 01:50 → 01:10 là **âm 40 phút**: khách "được trả trước khi đón".

Dấu vết DST trong tệp thật	Số chuyến
Đón 01:xx ngày 10/3, trả sau 03:00 (thời lượng dư một giờ)	1.093
Tháng 11: giờ trả sớm hơn giờ đón	1.078
... trong đó đón lúc 01:xx ngày 3/11	1.001

Độc bảng. Dòng 3 so với dòng 2: gần hết các chuyến "trả trước khi đón" của cả tháng 11 dồn vào đúng giờ lập. Đó là bằng chứng mạnh rằng cột thời gian là giờ đồng hồ New York, không phải UTC.

Tóm lại. Cột giờ của taxi New York là giờ địa phương naive. Đếm trên nó tạo một giờ 0 chuyến giả ngày 10/3/2024 và một giờ gấp đôi ngày 3/11/2024. Giờ lập không tách được thì ghi NaN ("không biết"), không ghi 0.

Tự kiểm tra. Một chuyến ghi đón 2024-03-10 01:45, trả 2024-03-10 03:10 (naive, giờ New York). Chuyến dài bao nhiêu phút?

Đáp án (tự làm trước rồi mới đọc)

Đón 01:45 EST = 06:45Z. Trả 03:10 EDT = 07:10Z. Chuyến dài **25 phút**. Nhầm hay gặp: trừ thẳng hai con số naive, ra 85 phút (dư một giờ vì 02:xx không tồn tại).

4.3 Đổi tần suất: khoảng nào, gộp thế nào, giờ trống ra sao

Vấn đề. Taxi là một chuỗi **không đều**: mỗi chuyến là một sự kiện ở một giây bất kỳ. Mô hình dự báo cần chuỗi **đều**: mỗi giờ đúng một con số. Muốn đổi, phải trả lời ba câu. Một sự kiện đúng lúc 10:00 thuộc giờ nào? Gộp các giá trị trong giờ bằng cộng hay trung bình? Giờ không có gì thì ghi gì?

Trực giác. Bảng chấm công theo ca. Ca sáng tính từ 9:00 tới **trước** 10:00. Người đến đúng 10:00 được tính vào ca 10:00. Ca được gọi tên bằng giờ bắt đầu. Đó chính là mặc định của pandas.

Khoảng và nhãn. Pandas chia trục thời gian thành các khoảng dài bằng tần suất. Với tần suất giờ, khoảng đầu là [9:00, 10:00). Dấu ngoặc vuông [nghĩa là **có** tính 9:00. Dấu ngoặc tròn) nghĩa là **không** tính 10:00. Khoảng như vậy gọi là "đóng bên trái" (`closed="left"`). Kết quả của khoảng được ghi tên bằng mốc đầu, 9:00 (`label="left"`).

Ví dụ số nhỏ — tự tính tay. Bốn chuyến lúc 9:00, 9:40, 10:00, 10:20. Đếm theo giờ bằng cách cộng số chuyến:

- Mặc định (đóng trái, nhãn trái): [9:00, 10:00) có 9:00 và 9:40 → **2**, tên "9:00". [10:00, 11:00) có 10:00 và 10:20 → **2**, tên "10:00".
- `closed="right", label="right"` (đóng phải, nhãn phải): khoảng thành (8:00, 9:00], (9:00, 10:00], (10:00, 11:00], đặt tên bằng mốc cuối. Chuyến 9:00 rơi vào (8:00, 9:00], tên "9:00". Chuyến 9:40 và 10:00 rơi vào (9:00, 10:00], tên "10:00". Kết quả {9:00: 1, 10:00: 2, 11:00: 1}.

Cùng dữ liệu, khác bảng. Đọc một bảng đã gộp mà không biết `closed` và `label` thì không biết con số thuộc khoảng nào.

Công thức. Với đóng trái, nhãn trái, mỗi thời điểm được đưa về mốc đầu khoảng của nó:

$$\text{nhãn}(t) = \left\lfloor \frac{t}{\Delta} \right\rfloor \times \Delta$$

- t : thời điểm của sự kiện.
- Δ : độ dài một khoảng (tần suất), ví dụ 1 giờ.
- $\lfloor x \rfloor$: làm tròn xuống số nguyên.

Nói bằng lời. Nhãn là thời điểm làm tròn **xuống** tới bội số gần nhất của tần suất. Với $\Delta = 1$ giờ, 9:40 là 9,67 giờ; làm tròn xuống được 9, rồi 9×1 giờ = 9 giờ, tức nhãn 9:00. Còn 10:00 đúng bằng 10 giờ, nên nhãn là 10:00, không phải 9:00. Trong pandas, phép này là `dt.floor("h")`.

Mặc định không giống nhau cho mọi tần suất. Mã tần suất của pandas: `h` giờ, `D` ngày, `W` tuần, `ME` cuối tháng, `QE` cuối quý, `YE` cuối năm. `BME`, `BQE`, `BYE` là ngày làm việc cuối cùng của tháng, quý, năm (chữ B là *business*). Với `W`, `ME`, `QE`, `YE` và ba mã B, pandas mặc định **đóng phải, nhãn phải**. Ví dụ doanh số tháng 1 được ghi nhãn 31/1. Mọi tần suất khác mặc định đóng trái, nhãn trái.

Tài liệu pandas cảnh báo về mặc định **đóng trái, nhãn trái** (dùng cho giờ, ngày): nhãn nằm ở **đầu** khoảng, nhưng con số của khoảng chỉ biết đủ ở **cuối** khoảng. Vì vậy dễ vô tình "nhìn trước": giá trị của các thời điểm sau bị gán cho nhãn sớm hơn. Trong dự báo, đó là **rò rỉ tương lai**.

Mượn trước — rò rỉ tương lai (buổi 5 và 13 học kỹ)

- **Rò rỉ tương lai:** mô hình được cho xem thông tin mà ở thời điểm dự báo thật chưa có. Kết quả chảm vì thế đẹp giả.
- Ví dụ: nhãn 09:00 (đóng trái) chứa số chuyển của cả khoảng 9:00–9:59. Con số đó chỉ biết đủ lúc 10:00. Nếu lúc 9:05 bạn dùng "giá trị nhãn 09:00" làm đầu vào, bạn đang dùng các chuyển lúc 9:40 chưa xảy ra.

Gộp bằng gì. Số lượng (số chuyển, kWh điện) thì **cộng**: giờ đó có tổng bao nhiêu. Số đo trạng thái (nhiệt độ, giá) thì lấy **trung bình** hoặc **giá trị cuối**. Cộng 60 lần đo nhiệt độ trong một giờ ra "1.500 độ", vô nghĩa. Lấy trung bình số chuyển thì ra "số chuyển trung bình mỗi phút", tức đã đổi đơn vị mà không ai hay.

Giờ trống. Ba sự kiện: 00:10 = 1, 00:50 = 2, 02:20 = 5. Giờ 01:00 không có gì. Kết quả (chạy thật, `đáp-an/vi_du_nho.py`):

Cách	Các mốc ra	Giá trị	Dùng khi
<code>resample("h").sum()</code>	00:00, 01:00, 02:00	3; 0; 5	đếm sự kiện: giờ trống là 0 chuyển
<code>resample("h").mean()</code>	00:00, 01:00, 02:00	1,5; NaN; 5	số đo trạng thái: giờ trống là "không đo"
<code>asfreq("h")</code>	00:10, 01:10, 02:10	1; NaN; NaN	không hợp với sự kiện
<code>.sum()</code> rồi <code>reindex</code> lên 00:00–04:00	00:00 ... 04:00	3; 0; 5; 0; 0	cần đủ mốc cho cả khoảng

Đọc bảng. So dòng 1 với dòng 2: cùng giờ 01:00, cộng cho 0 còn trung bình cho NaN. Vì trung bình của "không có gì" là không xác định. Dòng 3 cho thấy `asfreq` **không gộp**. Nó lấy mốc đầu tiên

(00:10) làm gốc, bước từng giờ, và chỉ lấy giá trị **đúng tại** mốc đó. Giá trị 00:50 bị bỏ; 01:10 và 02:10 không có sự kiện đúng lúc đó nên là NaN. Dòng 4: `resample` chỉ phủ từ sự kiện đầu tới sự kiện cuối. `reindex` đặt bảng lên một danh sách mốc cho trước. Mốc nào bảng chưa có thì thêm dòng mới với giá trị điền, ở đây là 0.

Tự viết bằng NumPy. Làm tròn xuống giờ rồi đếm:

```
import numpy as np
t = np.array(["2024-01-01T00:10", "2024-01-01T00:50", "2024-01-01T02:20"], dtype="datetime64[m]")
v = np.array([1, 2, 5])
gio = t.astype("datetime64[h]") # đổi sang độ chính xác giờ = làm tròn xuống giờ
luoi = np.arange(gio.min(), gio.max() + 1) # đủ mốc: 00, 01, 02
tong = np.array([v[gio == g].sum() for g in luoi]) # [3 0 5]
```

Thư viện. `s.resample("h").sum()` trên một Series có chỉ mục thời gian. Hoặc `df.groupby(df["t"].dt.floor("h"))`, rồi `.sum()`. Cách thứ hai không tự thêm giờ trống, phải `reindex` sau.

Dữ liệu thật. Code đầu buổi đếm tháng 3 bằng `resample("h")` trên giờ naive, được 744 giờ. Đếm đúng trên UTC chỉ có 743 giờ, vì ngày đổi giờ mùa xuân ngắn một giờ. Giờ thừa chính là giờ trống ảo ở mục 4.2.

Tóm lại. Mặc định pandas gộp theo khoảng đóng trái, nhãn là mốc đầu: [9:00, 10:00) tên "9:00". Số lượng thì cộng, trạng thái thì trung bình hoặc giá trị cuối. Giờ trống của đếm sự kiện là 0; giờ trống của số đo là NaN.

Tự kiểm tra. Nhiệt độ đo lúc 00:15 = 20 °C, 00:45 = 22 °C, 02:30 = 18 °C. Gộp theo giờ bằng `resample("h")` với cách gộp đúng. Kết quả giờ 00:00, 01:00, 02:00 là gì?

Đáp án (tự làm trước rồi mới đọc)

Nhiệt độ là trạng thái nên lấy trung bình: 00:00 → $(20 + 22) / 2 = \mathbf{21}$; 01:00 → **NaN** (không đo); 02:00 → **18**. Nhầm hay gặp: dùng `.sum()` ra 42 độ và 0 độ lúc 01:00 — "0 độ" là một nhiệt độ thật, rất khác "không đo".

4.4 Ghép hai nguồn: cùng UTC trước, đúng hướng thời gian sau

Vấn đề. Ghép số chuyển theo giờ với thời tiết theo giờ để hỏi "mưa có làm tăng khách không". Nếu hai bảng dùng hai trục thời gian khác nhau, mỗi dòng taxi sẽ nhận thời tiết của một giờ khác. Kết luận rút ra sẽ sai mà không có lỗi nào hiện lên.

Thời tiết Open-Meteo của buổi được tải theo UTC, có chú ý: nếu xin theo giờ New York, dịch vụ này dùng một offset cố định cho cả khoảng, sai quanh hai ngày đổi giờ (Đọc thêm).

Trực giác. Hai người hẹn gặp "lúc 8 giờ". Một người nghĩ theo giờ Hà Nội, người kia theo giờ UTC. Họ lỡ hẹn 7 tiếng mà cả hai đều đúng giờ theo đồng hồ của mình.

Ví dụ số nhỏ — tự tính tay. Code đầu buổi ghép bằng con số giờ: dòng taxi "15:00" (giờ New York, naive) nối với dòng thời tiết "15:00" (UTC). Ngày 15/3 New York đang EDT:

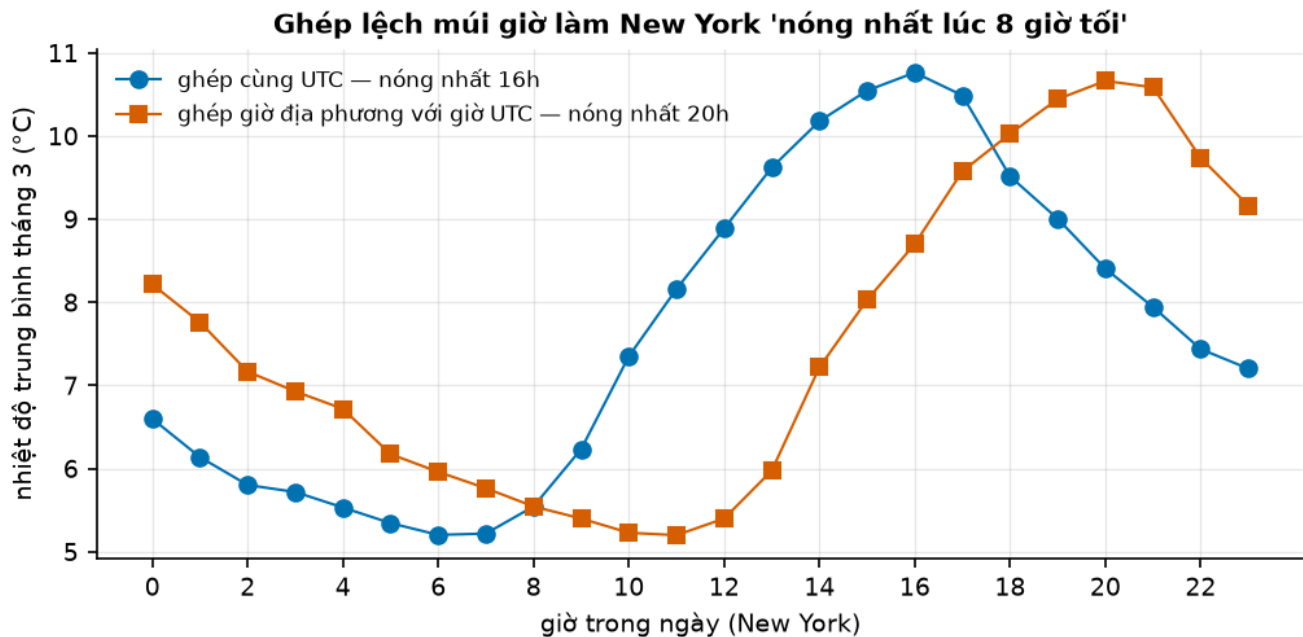
- Dòng thời tiết 15:00Z là lúc 15:00 − 4 = 11:00 giờ New York. Vậy dòng taxi 15:00 nhận thời tiết của **11:00**, sớm 4 giờ.
- Ngược lại, thời tiết của 16:00 New York là 16:00 + 4 = 20:00Z. Nó bị gán vào dòng taxi **"20:00"**.

- Mọi giá trị thời tiết bị đẩy **muộn** 4 giờ trên trục giờ New York. Nắng nhất lúc 16:00 sẽ hiện ra ở 20:00.

Độ lệch không cố định cả năm:

- Từ 10/3/2024 đến 3/11/2024 (EDT), hai bảng lệch 4 giờ.
- Trước 10/3/2024 và sau 3/11/2024 (EST), hai bảng lệch 5 giờ.
- Tháng 3 chỉ có 9 ngày đầu là EST, nên độ lệch 4 giờ chiếm phần lớn tháng.

Phép thử nhanh: **giờ nóng nhất trong ngày**. Không cần hiểu khí tượng, chỉ cần biết buổi chiều nóng hơn buổi tối.



Cách đọc hình.

1. **Trục ngang**: giờ trong ngày theo cột giờ của bảng ghép (0 tới 23 giờ).
2. **Trục dọc**: nhiệt độ trung bình của tháng 3/2024 ở giờ đó (°C).
3. **Ký hiệu**: đường xanh chấm tròn là bảng ghép đúng (cả hai về UTC, rồi đọc giờ theo New York). Đường cam chấm vuông là bảng của `code/` (giờ New York ghép với giờ UTC).
4. **Nhìn vào đâu**: đỉnh của mỗi đường. Đường xanh cao nhất lúc 16 giờ, thấp nhất lúc sáng sớm. Đường cam có cùng hình dạng nhưng trượt sang phải khoảng 4 giờ, nên đỉnh rơi vào 20 giờ và đáy rơi vào gần trưa.
5. **Kết luận**: ghép lệch múi giờ làm New York "nóng nhất lúc 8 giờ tối", một điều vô lý dễ thấy.

Muộn trước — tương quan (buổi 2 và 8 học kỹ)

- **Tương quan** (hệ số r) là một số từ -1 đến 1 , đo hai đại lượng cùng tăng giảm tới đâu. Gần 1 : cùng lên cùng xuống. Gần -1 : cái này lên thì cái kia xuống. Gần 0 : không thấy quan hệ.
- Ví dụ $x = 1, 2, 3$ và $y = 2, 4, 6$ thì $r = 1$. Đổi $y = 6, 4, 2$ thì $r = -1$.
- Cách tính: lấy độ lệch khỏi trung bình của từng số, d_x và d_y . Khi đó $r = \frac{\sum d_x d_y}{\sqrt{\sum d_x^2 \cdot \sum d_y^2}}$.
Với ví dụ trên: $d_x = -1, 0, 1$ và $d_y = -2, 0, 2$, nên $r = 4/\sqrt{2 \times 8} = 1$.
- Tương quan không chứng minh cái này gây ra cái kia.

Mượn trước — khử mùa vụ (buổi 6 học kỹ)

- **Mùa vụ:** nhịp lặp lại đều đặn. Taxi đông lúc 8 giờ sáng ngày thường, vắng lúc 4 giờ sáng, tuần nào cũng vậy.
- **Khử mùa vụ** theo giờ trong tuần: lấy số chuyến trừ đi mức trung bình của **cùng giờ, cùng thứ**. Phần còn lại cho biết giờ này đông hơn hay vắng hơn bình thường.
- Ví dụ: 8 giờ sáng thứ Hai trung bình 5.000 chuyến; hôm nay 5.400 chuyến; phần còn lại là $5.400 - 5.000 = +400$.
- Phải khử trước khi tính tương quan với mưa. Nếu không, tương quan sẽ bị nhịp ngày đêm chi phối: đêm vắng khách và đêm cũng có thể hay mưa, dù mưa không liên quan gì.

Vì sao ghép lệch làm đổi dấu tương quan. Ví dụ 8 giờ liền nhau:

- Mưa (mm): 0, 0, 2, 2, 0, 0, 0, 0.
- Số chuyến sau khi khử mùa vụ, tính bằng trăm chuyến so với mức bình thường: $-1, -1, +3, +3, -1, -1, -1, -1$.
- Ghép đúng: hai giờ mưa chính là hai giờ đông hơn bình thường, nên $r = 1$.
- Ghép lệch 4 giờ: mưa bị dời xuống hai giờ cuối, thành 0, 0, 0, 0, 0, 0, 2, 2. Hai giờ đông khách thật giờ không có mưa. Mưa lại nằm ở hai giờ vắng hơn bình thường. Tính lại ra $r \approx -0,33$: dấu đã đổi ([dap-an/vi_du_nho.py](#)).

Trên dữ liệu thật tháng 3/2024, tương quan giữa lượng mưa và số chuyến cùng giờ (đã khử mùa vụ giờ trong tuần):

Cách ghép	Tương quan mưa - số chuyến
lệch (giờ New York naive với giờ UTC)	$-0,100$
đúng (cả hai về UTC)	$+0,136$

Độc bảng. Hai dòng khác cả dấu. Mô hình học từ bảng ghép lệch sẽ "biết" rằng mưa làm ít người đi taxi, ngược với bảng ghép đúng. Không có dòng lỗi nào báo điều này; chỉ phép thử giờ nóng nhất bắt được.

Khi hai nguồn không cùng nhịp: `merge_asof`. Có lúc bảng bên phải không có đúng mốc của bảng bên trái. Ví dụ: giá chỉ được ghi lại mỗi khi thay đổi. `merge_asof` ghép mỗi dòng bên trái với một dòng bên phải **gần nhất theo thời gian**. Tham số `direction` chọn hướng; mặc định `"backward"` là nhìn về **quá khứ**. Ví dụ:

Bảng bên phải: thời điểm	09:30	10:30	11:00
giá	9	10	11

Bên trái	"backward" (mặc định)	"forward"	<code>allow_exact_matches=False</code>
10:00	9 (từ 09:30)	10 (từ 10:30)	9 (từ 09:30)
11:00	11 (từ 11:00, đúng lúc)	11 (từ 11:00)	10 (từ 10:30)

Độc bảng. Cột `"forward"` cho dòng 10:00 nhận giá lúc 10:30, tức kéo tương lai về: rò rỉ. Cột cuối cấm lấy dòng **đúng cùng** thời điểm, nên 11:00 nhận giá lúc 10:30. Dùng khi con số ghi nhãn 11:00 thật ra được công bố trễ vài phút sau 11:00. Đúng lúc 11:00 bạn chưa có nó.

Tóm lại. Đưa mọi nguồn về UTC có ghi múi giờ rồi mới ghép. Ghép lịch không báo lỗi; nó đòi dữ liệu và có thể làm đổi dấu tương quan. Kiểm nhanh bằng giờ nóng nhất. Khi ghép theo thời gian gần nhất, chỉ nhìn về quá khứ (backward).

Tự kiểm tra. Ngày 20/11/2024, bảng của `code/` ghép dòng taxi "15:00" (giờ New York) với dòng thời tiết "15:00" (UTC). Dòng taxi đó nhận thời tiết của mấy giờ New York?

Đáp án (tự làm trước rồi mới đọc)

Sau 3/11/2024 New York là EST (UTC−5). Đổi 15:00Z về New York: $15 - 5 = 10$, tức **10:00**, sớm 5 giờ. Nhầm hay gặp: nhớ "lịch 4 giờ" cho cả năm. Độ lệch 4 giờ chỉ đúng từ 10/3 đến 3/11/2024.

4.5 Dạng dài: một chuỗi cho mỗi khu vực

Vấn đề. Không chỉ đếm cả thành phố. Ta muốn một chuỗi theo giờ cho **từng khu vực đón khách**. Tháng 3 có 259 khu vực có chuyến (bảng tra có 265). Lưu 259 chuỗi thế nào cho gọn, và để các buổi sau cắt cùng mốc?

Trực giác. Số bán hàng ghi mỗi dòng "cửa hàng nào, ngày nào, bán bao nhiêu" thì thêm cửa hàng mới không phải thêm cột.

Ví dụ số nhỏ — tự tính tay. Hai khu vực A, B, ba giờ.

unique_id	ds	y
A	05:00Z	3
A	06:00Z	0
A	07:00Z	5
B	05:00Z	1
B	06:00Z	2
B	07:00Z	0

Đọc bảng. Đây là **dạng dài**: $2 \times 3 = 6$ dòng, mỗi (chuỗi, giờ) một dòng. Cột `unique_id` là tên chuỗi. Cột `ds` (viết tắt của *datestamp*, dấu thời gian) là thời điểm. Cột `y` là giá trị cần dự báo. **Dạng rộng** của cùng dữ liệu có 3 dòng (mỗi giờ một dòng) và 2 cột số (A, B): dòng 05:00Z là A = 3, B = 1. Các thư viện Nixtla (bộ thư viện dự báo mã nguồn mở, như statsforecast và mlforecast, dùng ở các giai đoạn sau của khoá) và phần lớn khoá này nhận dạng dài.

Lưới chung. Mọi chuỗi phải có **cùng một danh sách mốc**, gọi là lưới chung. Khu vực vắng khách lúc 3 giờ sáng vẫn cần dòng "3 giờ sáng, 0 chuyến". Nhờ vậy ta so được giữa các chuỗi, và các buổi sau làm **backtest** cùng mốc cắt cho mọi chuỗi. Lưới tháng 3/2024 dựng như sau:

- Đầu tháng: 00:00 ngày 1/3/2024 giờ New York là EST, tức 05:00Z.
- Cuối tháng: 00:00 ngày 1/4/2024 giờ New York là EDT, tức 04:00Z.
- Tham số `khoang=("2024-03-01 05:00", "2024-04-01 04:00")` hiểu là khoảng nửa mở [05:00Z ngày 1/3/2024, 04:00Z ngày 1/4/2024): **có** mốc đầu, **không có** mốc cuối.
- Mốc cuối cùng của lưới là 03:00Z ngày 1/4/2024, tức 23:00 EDT ngày 31/3/2024. Đếm ra **743** mốc. Nếu tính cả mốc cuối thì sẽ là 744, dư một giờ thuộc tháng 4.

```
dai = chuan_hoa_thoi_gian(chuyen.assign(mot=1),          # mỗi chuyến một cột "mot" = 1, cộng lại
                           thành số chuyến
                           "tpep_pickup_datetime", "mot", "h",
                           mui_gio_nguon="America/New_York", cot_id="PULocationID", gop="sum",
                           mo_ho="NaT",                  # giờ lập 3/11 không tách được → bỏ, mốc
liên quan ghi NaN
                           khoang=("2024-03-01 05:00", "2024-04-01 04:00")) # lưới chung [đầu,
cuối)
dai.head(2)
# unique_id          ds      y
#      1 2024-03-01 05:00:00+00:00  0.0
#      1 2024-03-01 06:00:00+00:00  0.0
rong = dai.pivot(index="ds", columns="unique_id", values="y") # đổi sang dạng rộng: (743, 259)
```

pivot đổi dạng dài sang dạng rộng: giá trị của cột `index` thành dòng, giá trị của cột `columns` thành cột. Kết quả có 743 dòng (giờ) và 259 cột (khu vực).

Dữ liệu thật. Bảng dạng dài có 192.437 dòng, và **53,7%** số ô bằng 0: đa số khu vực hiếm khi có khách trong một giờ. Trong khi đó vài khu vực rất đông:

Khu vực đông nhất tháng 3/2024	Số chuyến
Midtown Center	163.267
sân bay quốc tế John F. Kennedy	157.703
Upper East Side South	155.631

Độc bảng. Ba khu vực này mỗi nơi hơn 150 nghìn chuyến một tháng, trong khi hơn một nửa số ô của bảng là 0. Tổng rất lớn ở vài chuỗi, rất thưa ở phần còn lại: hình dạng điển hình của dữ liệu nhiều chuỗi mà buổi 19 và 22 xử lý.

Lưu ý: cột của bảng rộng xếp 1, 10, 100, 101..., không phải 1, 2, 3. `unique_id` là chuỗi ký tự nên được sắp như trong từ điển ("10" đứng trước "2"). Muốn sắp theo số thì đổi kiểu trước khi `pivot`.

Tóm lại. Dạng dài `unique_id`, `ds`, `y` lưu mỗi (chuỗi, mốc) một dòng. Mọi chuỗi trải lên cùng một lưới mốc; tháng 3/2024 theo UTC có 743 giờ, không phải 744.

Tự kiểm tra. 3 cửa hàng, mỗi cửa hàng bán theo giờ trong 2 ngày thường (không đổi giờ). Dạng dài có bao nhiêu dòng? Dạng rộng có mấy dòng, mấy cột số?

Đáp án (tự làm trước rồi mới đọc)

Mỗi chuỗi $2 \times 24 = 48$ mốc. Dạng dài: $3 \times 48 = \mathbf{144}$ dòng. Dạng rộng: **48 dòng, 3 cột số**. Nhầm hay gặp: quên rằng giờ không bán gì vẫn phải có dòng ($y = 0$) trên lưới chung.

Nâng cao — ba công cụ và hai bản pandas

Nâng cao — có thể bỏ qua lần đọc đầu. Cả mục này là kiến thức thêm. Lab chỉ cần ý chính của mục 4.1: DuckDB không báo lỗi ở giờ mùa hè và lấy múi giờ theo máy.

pandas, polars, DuckDB. Cùng một việc, ba cách. Thư viện pandas làm kiểu *eager* (chạy ngay): mỗi lệnh chạy xong mới tới lệnh sau, cả bảng nằm trong RAM. Thư viện polars làm được cả *eager* lẫn *lazy*

(chạy lười). Với lazy, bạn viết cả chuỗi lệnh trước; polars xem toàn bộ rồi mới chạy, nhờ vậy nó tự bỏ các cột không dùng. DuckDB chạy SQL thẳng trên tệp Parquet hay CSV.

Việc	pandas	polars	DuckDB
Gộp theo giờ	<code>dt.floor("h")</code>	<code>dt.truncate("1h")</code>	<code>date_trunc('hour', ts)</code>
Giờ không tồn tại (02:30 ngày 10/3/2024)	báo lỗi <code>nonexistent time</code>	báo lỗi; tham số <code>non_existent</code> chỉ có <code>"raise"</code> (báo lỗi) hoặc <code>"null"</code> (để trống)	không báo lỗi , ra 07:30Z (trùng 03:30 EDT)
Giờ lặp (01:30 ngày 3/11/2024)	báo lỗi <code>Cannot infer dst time</code>	báo lỗi; <code>ambiguous = "earliest"</code> (lần sớm, EDT), <code>"latest"</code> (lần muộn, EST) hoặc <code>"null"</code>	không báo lỗi , chọn EST (06:30Z)

Độc bảng. Hai dòng cuối là chỗ quan trọng. pandas và polars dừng lại và bắt bạn chọn. DuckDB tự chọn mà không nói gì.

DuckDB còn một bẫy. Nó đọc và hiển thị thời gian có múi giờ theo tham số `TimeZone` của phiên làm việc. Mặc định tham số này lấy múi giờ của máy: máy ở Việt Nam là `Asia/Ho_Chi_Minh`, máy ở New York là `America/New_York`. Cùng một câu lệnh có thể ra bảng khác nhau trên hai máy. Vì vậy luôn viết `SET TimeZone = 'UTC'` ở đầu script.

pandas 2 và pandas 3. Buổi 1–13 dùng pandas 3; các buổi dùng statsforecast, mlforecast dùng pandas 2.3.3 vì các thư viện đó chưa hỗ trợ pandas 3. Hàm của buổi này chạy đúng trên cả hai (bộ chấm đã chạy qua cả hai bản). Hai chỗ code cũ hay hỏng khi lên pandas 3: cột lẫn nhiều offset mà thiếu `utc=True` thì báo lỗi `Mixed timezones detected`; mã tần suất `"H"` (chữ hoa) báo lỗi, phải viết `"h"`. Bảng khác biệt đầy đủ ở Phụ lục A của khoá.

5. Lab từng bước

Mọi lệnh chạy trong thư mục `lab/`. Tiễn tố `env -u VIRTUAL_ENV uv run --no-sync --project 00-nen` nghĩa là "chạy bằng Python của nền buổi này, không cài thêm gì".

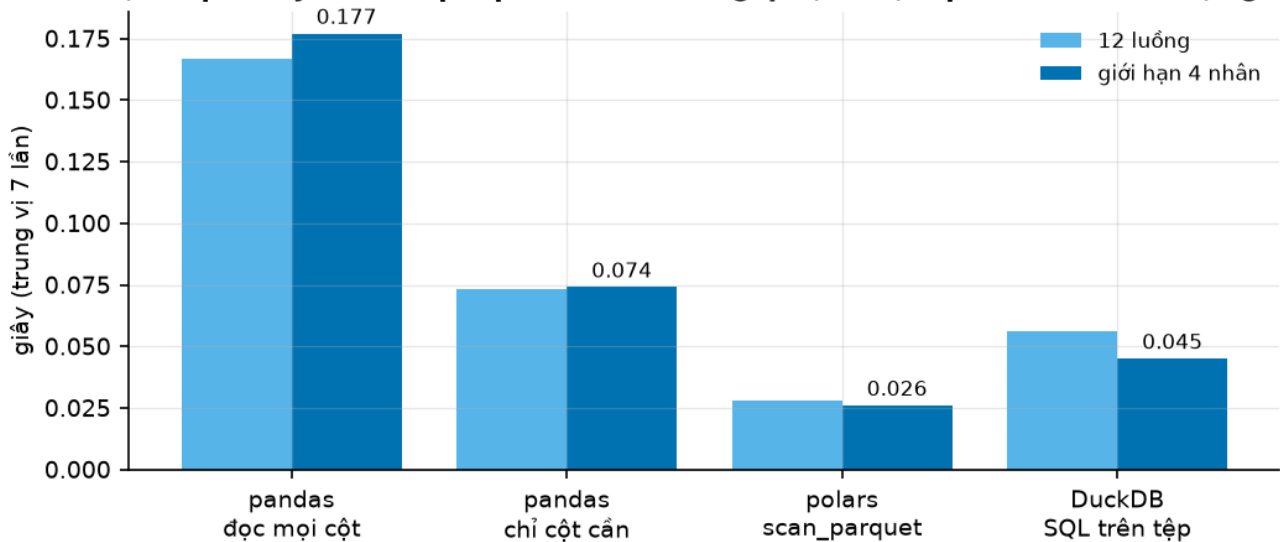
Bước 1 — Dựng nền, bấm giờ

Mục đích: có môi trường và dữ liệu; đo xem máy bạn đọc 3,6 triệu chuyển nhanh tới đâu bằng ba công cụ.

```
cd lab && make up
env -u VIRTUAL_ENV uv run --no-sync --project 00-nen python ../code/bam_gio.py
```

Script chạy mỗi cách **1 lần làm nóng** rồi **7 lần đo**, báo **trung vị** của 7 lần (xếp 7 lần đo tăng dần, lấy lần đúng giữa — lần thứ 4). Lần làm nóng để hệ điều hành nạp tệp vào bộ nhớ đệm và thư viện khởi tạo xong. Lấy trung vị để một lần chạy chậm bất thường không kéo lệch kết quả.

3,6 triệu chuyển: chỉ đọc cột cần đã nhanh gấp 2,4 lần; mọi cách đều dưới 0,2 giây



Cách đọc hình.

1. **Trục ngang:** bốn cách làm cùng một việc (đọc tệp tháng 3 rồi đếm chuyển theo giờ).
2. **Trục dọc:** thời gian chạy (giây, trung vị 7 lần).
3. **Ký hiệu:** cột xanh nhạt là máy soạn khoá dùng đủ 12 luồng; cột xanh đậm là giới hạn 4 nhân cho sát máy học viên, số ghi trên cột.
4. **Nhìn vào đâu:** so cột "pandas đọc mọi cột" với "pandas chỉ cột cần".
5. **Kết luận:** chỉ đọc cột cần đã nhanh gấp 2,4 lần; mọi cách đều dưới 0,2 giây.

Đọc kết quả:

- Ghi số luồng CPU và bốn con số của máy bạn, so với hình. Tệp Parquet lưu theo cột, nên đọc một cột thì bỏ qua được phần lớn tệp. Ở cỡ dữ liệu này, khác biệt giữa các công cụ chỉ là phần trăm giây.
- Mọi cách đều báo **747** nhóm giờ, không phải 744 giờ của tháng 3. Phép đếm khớp như sau (chạy thật trên tệp):

Phần	Số chuyển	Số nhóm giờ
chuyển đón trong tháng 3	3.582.605	743
chuyển đón ngoài tháng 3 (sớm nhất 2002-12-31 22:17:10)	23	4
cả tệp	3.582.628	747

Đọc bằng. Trong tháng chỉ có 743 nhóm, không phải 744. Giờ 02:00 ngày 10/3 không tồn tại nên không có chuyển nào. `groupby` không tạo nhóm cho giờ không có dòng. 23 chuyển lạ rơi vào 4 giờ lạ, nên $743 + 4 = 747$. Con số 3.582.605 ở Bước 4 chính là tổng sau khi bỏ 23 chuyển này.

Bước 2 — Đếm theo giờ bằng code có sẵn, hỏi năm câu

Mục đích: thấy tận mắt giờ 0 chuyển và giờ gấp đôi, rồi học năm câu chuẩn đoán trước khi tin một cột thời gian.

```
env -u VIRTUAL_ENV uv run --no-sync --project 00-nen python ../code/thoi_gian.py
```

Output: 3582605 chuyến → 744 giờ và giờ nóng nhất: 20. Tụ in các giờ 00:00–05:00 của ngày 10/3/2024 và 3/11/2024 từ `dem_chuyen_theo_gio`. Bạn phải thấy giờ 0 chuyến và giờ gần gấp đôi như hình mục 4.2.

Sau đó chạy năm câu hỏi trên tệp tháng 11 (`t` là bảng đọc bằng `pd.read_parquet`). Output thật ghi ở comment:

```
don, tra = t["tpep_pickup_datetime"], t["tpep_dropoff_datetime"]
don.dtype, don.dt.tz          # 1. kiểu, múi giờ          → datetime64[us], None (naive!)
don.min(), don.max()          # 2. khoảng thời gian      → 2002-12-31 22:17:43, 2024-12-01
22:04:33
(~don.between("2024-11-01", "2024-12-01", inclusive="left")).sum() # 3. ngoài tháng → 50
(tra < don).sum()              # 4. thời lượng âm          → 1078
gio = don.dt.floor("h").drop_duplicates()                          # mỗi giờ có dữ liệu giữ một
lần
gio[gio.between("2024-11-01", "2024-12-01", inclusive="left").dt.date.value_counts().value_counts()
# 5. số giờ có dữ liệu/ngày → {24: 30}
```

Câu 5 đếm hai lần:

- `.dt.date.value_counts()` đếm **mỗi ngày có bao nhiêu giờ có chuyến**.
- `.value_counts()` lần hai đếm **bao nhiêu ngày có cùng số giờ đó**.
- Ví dụ nhỏ: ba ngày lần lượt có 3, 2, 3 giờ. Lần đếm thứ hai cho `{3: 2, 2: 1}`: hai ngày có 3 giờ, một ngày có 2 giờ.
- Vậy `{24: 30}` nghĩa là "30 ngày, ngày nào cũng có đủ 24 giờ".

Độc kết quả:

- Tệp tháng 3: câu 5 cho `{24: 30, 23: 1}`, và ngày chỉ có 23 giờ là 10/3/2024. Giờ bị mất lộ ra ngay.
- Tệp tháng 11: câu 5 cho `{24: 30}`. Ngày 3/11/2024 có 25 giờ thật, nhưng trên giờ naive chỉ còn 24: **không có dấu hiệu gì**.
- Giờ lập chỉ lộ ra qua số chuyến bất thường của giờ 01:00, hoặc qua câu 4 (thời lượng âm dồn vào giờ đó). Đếm mốc thời gian không thay được việc biết múi giờ của nguồn.
- Viết một câu cho mỗi ngày: đây là lỗi của khách đi taxi hay lỗi của trục thời gian?

Bước 3 — Ghép thời tiết, thấy "nóng nhất lúc 20h"

Mục đích: hiểu vì sao `gio_nong_nhat()` ra 20 trước khi sửa.

Mở `code/thoi_gian.py`, đọc `doc_thoi_tiet` và `ghép_thoi_tiet`. Cột `time` của thời tiết là UTC. Cột giờ của taxi là giờ New York. Hàm ghép nối hai bảng theo con số giờ, không hỏi múi giờ.

Độc kết quả: giải thích vì sao đường cam trượt **khoảng 4** giờ chứ không phải 5. Gợi ý: phần lớn tháng 3/2024 nằm sau ngày đổi giờ mùa xuân, khi New York là EDT (mục 4.4). Nếu bạn giải thích bằng "New York luôn lệch 4 giờ" thì đọc lại mục 4.1.

Bước 4 — Sửa `chuan_hoa_thoi_gian`

Mục đích: viết lại hàm cột mốc M0 cho đúng đặc tả dưới đây, đến khi `make check` xanh.

Đặc tả các tham số (cũng chép trong docstring của hàm trong `code/thoi_gian.py`):

Tham số	Nghĩa	Mặc định
tan_suat	tần suất của kết quả, mã pandas ("h" , "D" ...)	"h"
mui_gio_nguon	múi giờ của cột thời gian khi nó naive ; cột có offset thì bỏ qua	"UTC"
cot_id	cột tên chuỗi; <code>None</code> = chỉ một chuỗi, tên "chuoi"	None
gop	"sum" , "count" cho số lượng/sự kiện; "mean" , "last" cho trạng thái	"sum"
mo_ho	xử lý giờ lặp: "raise" , "infer" , "NaT" hoặc mảng True/False (xem mục 4.1)	"raise"
dien	giá trị cho mốc không có dữ liệu; <code>None</code> = 0 với sum/count, NaN với mean/last	None
bo_trung	bỏ dòng trùng hết (cùng chuỗi, thời điểm, giá trị); <code>None</code> = bật cho mean/last, tắt cho sum/count	None
khoang	(bắt đầu, kết thúc) UTC: lưới chung [bắt đầu, kết thúc]; <code>None</code> = mỗi chuỗi từ mốc đầu tới mốc cuối của nó	None

Kết quả: bảng ba cột `unique_id`, `ds`, `y`; `ds` là thời gian UTC có múi giờ; `df.attrs` ghi số dòng bỏ vì giờ mùa hè và số dòng trùng. Làm theo thứ tự:

- Mọi đầu vào về UTC có múi giờ. Cột có offset (ví dụ `+07:00` hay `Z`): `pd.to_datetime(..., utc=True)`. Cột naive: `tz_localize(mui_gio_nguon, ambiguous=mo_ho, nonexistent="NaT")` rồi `tz_convert("UTC")`.
- Bỏ dòng trùng **chỉ** với dữ liệu trạng thái hoặc khi `bo_trung=True`. Với sự kiện, hai chuyển cùng giây là hai chuyển thật: bật bỏ trùng trên taxi sẽ xóa hơn 1,8 triệu chuyển (đã thử).
- `dt.floor(tan_suat)` rồi gộp theo `gop`.
- `reindex` mỗi chuỗi lên lưới đủ mốc. Giờ trống ghi 0 với sự kiện, NaN với trạng thái. Mốc mà dòng mơ hồ có thể thuộc về thì ghi NaN; cách tìm các mốc đó ở mục 4.2 (danh sách "ba việc", việc 2).

Sửa thêm `ghep_thoi_tiet` để **từ chối** (báo `ValueError`) thời gian naive, và `gio_nong_nhat` để lấy giờ theo múi giờ New York (`dt.tz_convert("America/New_York").dt.hour`).

```
make check
env -u VIRTUAL_ENV uv run --no-sync --project 00-nen python -m pytest cham -k dst -x
```

Lệnh thứ hai chạy riêng một nhóm test: `-k dst` chỉ chạy test có chữ "dst" trong tên; `-x` dừng ngay ở test hỏng đầu tiên để bạn đọc thông báo lỗi.

Độc kết quả:

- Xanh 11/11 là xong.
- Test `test_taxi_thang_3_khong_co_gio_0_chuyen_ao` còn đỏ: bạn vẫn đếm trên giờ naive.
- Test `test_ghep_thoi_tiet_dung_mui_gio` báo 20: `gio_nong_nhat` chưa đổi về giờ New York.
- Tổng số chuyển tháng 3 phải còn đúng 3.582.605. Thấp hơn nhiều thì bạn đã bỏ trùng trên dữ liệu sự kiện.

Bước 5 — Dạng dài theo khu vực

Mục đích: dựng bảng 259 chuỗi trên lưới chung như mục 4.5.

Gọi `chuan_hoa_thoi_gian(..., cot_id="PULocationID", khoang=("2024-03-01 05:00", "2024-04-01 04:00"))` đúng như đoạn code ở mục 4.5, để mọi khu vực cùng lưới. (`dem_chuyen_theo_gio(chuyen, theo_khu_vuc=True)` không truyền `khoang`, nên mỗi khu vực chỉ phủ từ chuyển đầu tới chuyển cuối của nó.)

Độc kết quả:

- Mỗi chuỗi có 743 dòng. Thấy 744 thì bạn đang tính cả mốc cuối của `khoang`, hoặc vẫn chạy trên giờ naive.
- `dai.duplicated(["unique_id", "ds"]).sum()` phải bằng 0: không có cặp (khu vực, giờ) nào lặp.
- Tỷ lệ ô bằng 0 khoảng 53,7%, như mục 4.5.

Bước 6 — Làm lại một bước bằng polars hoặc DuckDB

Mục đích: thấy mỗi công cụ xử lý giờ lặp khác nhau, và công cụ nào im lặng dờ dể dữ liệu.

Đếm chuyển theo giờ UTC ngày 3/11/2024, lọc 00:00–04:00 giờ New York, bằng polars hoặc DuckDB.

Với polars, dùng `dt.replace_time_zone("America/New_York", ambiguous="null")` rồi `dt.convert_time_zone("UTC")` và `dt.truncate("1h")`. Với DuckDB, câu lệnh đã chạy trên máy soạn khoá:

```
SET TimeZone = 'UTC'; -- không để DuckDB lấy múi giờ của máy
SELECT date_trunc('hour', timezone('America/New_York', tpep_pickup_datetime)) AS gio_utc, count(*)
FROM 'yellow_tripdata_2024-11.parquet'
WHERE tpep_pickup_datetime >= '2024-11-03 00:00' AND tpep_pickup_datetime < '2024-11-03 04:00'
GROUP BY 1 ORDER BY 1;
-- 04:00+00 → 7395 | 06:00+00 → 9869 | 07:00+00 → 2733 | 08:00+00 → 1900
```

Giờ UTC	polars (<code>ambiguous="null"</code>)	DuckDB
04:00Z	7.395	7.395
05:00Z	—	mất
06:00Z	—	9.869
nhóm <code>null</code>	9.869	—
07:00Z	2.733	2.733
08:00Z	1.900	1.900

Độc kết quả (đọc bảng): polars buộc bạn chọn. `ambiguous="null"` gom 9.869 chuyển vào một nhóm `null`, thấy ngay. DuckDB không hỏi: cả 9.869 chuyển bị dồn vào 06:00Z, còn 05:00Z biến mất khỏi bảng, không một dòng cảnh báo. Bảng DuckDB trông "sạch" nhất, và sai nhất. Hàm pandas của đáp án cho 05:00Z và 06:00Z là NaN, và ghi 9.869 dòng bị bỏ vào `attrs`.

6. Lỗi thường gặp & cách chẩn đoán

Triệu chứng	Nguyên nhân	Chẩn đoán	Sửa
Một giờ bằng 0 vào rạng sáng Chủ nhật tháng 3	đếm theo giờ trên giờ địa phương naive; giờ đó không tồn tại	in 00:00–05:00 ngày đổi giờ	gắn múi giờ rồi đổi UTC trước khi gộp
Một giờ gần gấp đôi rạng sáng Chủ nhật tháng 11	hai giờ thật gộp làm một	so với cùng giờ tuần sau	như trên; giờ không tách được → NaN
Chuyến có thời lượng âm	giờ lặp ghi naive	đếm <code>dropoff < pickup</code> theo giờ	tính thời lượng trên thời gian đã đổi UTC

Triệu chứng	Nguyên nhân	Chẩn đoán	Sửa
"Nóng nhất lúc 20h", mưa làm giảm khách	ghép giờ địa phương với giờ UTC	giờ nóng nhất trung bình	cả hai nguồn về UTC trước khi merge
Open-Meteo lệch 1 giờ quanh ngày đổi giờ	tham số <code>timezone=</code> dùng một offset cố định	so với bản tải <code>timezone=UTC</code>	luôn tải UTC
Kết quả DuckDB khác nhau giữa hai máy	<code>TimeZone</code> mặc định theo hệ điều hành	<code>SELECT current_setting('TimeZone')</code>	<code>SET TimeZone='UTC'</code> đầu script
<code>ValueError: Mixed timezones detected (pandas 3)</code>	nhiều offset lẫn trong một cột	xem giá trị có <code>+07:00</code> và <code>Z</code>	<code>pd.to_datetime(..., utc=True)</code>
Sau gộp, số chuyển giảm hàng triệu	bỏ dòng trùng trên dữ liệu sự kiện	so tổng trước/sau	chỉ bỏ trùng với trạng thái hoặc nguồn xuất trùng

7. Bài tập về nhà

- Tháng 3 và tháng 11 với polars thuần.** Viết lại `dem_chuyen_theo_gio` chỉ bằng polars lazy. Kết quả phải khớp pandas từng giờ, trừ các giờ mơ hồ; nộp bảng so sánh và thời gian chạy.
- Mùa đông lệch mấy giờ?** Dùng hai bộ đã có sẵn sau `make up: du-lieu/raw/nyc-tlc-yellow-2024-01/` và `du-lieu/raw/open-meteo-new-york-2024-01/` (thời tiết tháng 1, UTC). Lập lại phép thử giờ nóng nhất với bản ghép lệch và bản ghép đúng. Dự đoán trước con số rồi mới chạy.
- Một nguồn dữ liệu của bạn.** Lấy một bảng có cột thời gian từ công việc thật. Ghi: naive hay có múi giờ, nhãn là đầu hay cuối khoảng, gộp bằng cộng hay trung bình. Rồi chạy `chuan_hoa_thoi_gian` với tham số tương ứng.

8. Tiêu chí "Xong khi"

- [] `make check` xanh 11/11 với `code/` đã sửa.
- [] Trả lời bằng số: ngày 10/3 và 3/11/2024 bản naive sai ở giờ nào, sai bao nhiêu chuyển.
- [] Viết được bảng đối chiếu giờ New York ↔ UTC cho hai ngày đổi giờ mà không nhìn tài liệu.
- [] Giải thích được vì sao ghép lệch múi giờ làm tương quan mưa – số chuyển đổi dấu, và phép thử "giờ nóng nhất" bắt được nó.
- [] Có bảng dạng dài theo khu vực, lưới chung 743 giờ UTC, không dòng trùng.
- [] Ghi được con số bấm giờ của máy mình và nói vì sao chỉ đọc cột cần lại nhanh hơn.

9. Đọc thêm

- Hyndman, R.J., Athanasopoulos, G. et al. *Forecasting: Principles and Practice, the Pythonic Way*, chương 2 (chuỗi thời gian và tần suất) — <https://otexts.com/fpppy/>
- pandas User Guide — *Time series / date functionality* (múi giờ, DST, `resample`, `asfreq`): https://pandas.pydata.org/docs/user_guide/timeseries.html
- Python — `zoneinfo` (giờ lập và thuộc tính `fold`): <https://docs.python.org/3/library/zoneinfo.html>
- polars — `Expr.dt.replace_time_zone`: https://docs.pola.rs/api/python/stable/reference/expressions/api/polars.Expr.dt.replace_time_zone.html
- DuckDB — *TIMESTAMPTZ functions*: <https://duckdb.org/docs/current/sql/functions/timestamptz.html>

- NYC TLC — Data Dictionary Yellow Taxi: https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf
- Open-Meteo issue #1764 (offset múi giờ): <https://github.com/open-meteo/open-meteo/issues/1764>
- Phụ lục A của khoá — cú pháp thời gian pandas 2/3, polars, DuckDB (không bắt buộc).